Subiecte date

- Divide et impera
    - S3: al K-lea cel mai mic număr dintr-o secvență de N numere (S3.4 2020)
- Greedy
    - S2: condițiile care trebuie satisfăcute + exemplificare pentru construcția arborilor Huffman (S2.1 2019CB)
    - S2: exemplu de problemă care se poate rezolva și cu greedy și cu dinamică + diferențe, asemănări între algoritmi (inclusiv complexitate) (S2.1 2017CB)
    - S2: comparație cu algoritmii de programare dinamică + 3 exemple/categorie (S2.3 2017CC)
- Programarea dinamică
    - S1: de calculat numărul minim de înmulțiri scalare pentru înmulțirea a 5 matrici (++ parantezarea) (S1.5 2019CB)
- Backtracking & propagarea restricțiilor
    - S2: AC1 vs AC3 (asemănări și deosebiri) + analiză utilitate, funcționare, complexitate + de ce unul e mai ineficient și cum se poate rezolva problema (S2.3 2019CB)
    - S2: ce schemă de algoritm poate fi folosită pentru a determina dacă un careu de rebus are o asignare posibilă de cuvinte (S2.2 2017CA)
- **Parcurgeri**
    - S3: afișarea tuturor drumurilor distincte între 2 noduri dintr-un graf neorientat aciclic (S3.2 2020)
- **DFS**
    - **S1: parcurgere ++ timpi de descoperire și finalizare (S1.1 2020, S1.1 2019CB, S1.1 2017CA, S1.1 2017CB, S1.1 restanță)**
    - S2: comparație cu A* (S2.3 2017CA)
- **BFS**
    - **S1: parcurgere (S1.2 2020, S1.1 2017CC)**
    - S3: problema cu Facebook și găsirea celui mai scurt lanț de la un Gigel la un nou prieten (S3.1 2020)
- **Sortare topologică**
    - S3: calcularea numărului total de drumuri distincte între 2 noduri dintr-un graf orientat aciclic (S3.5 2020)
    - **S1: de efectuat (S1.2 2017CC)**
- **Kahn**
    - **S1: de efectuat (S1.2 2019CB)**
- **Componente tare conexe**
    - S1: determinarea lor prin parcurgerea grafului transpus ++ calcul timpi de descoperire & finalizare (S1.2 2017CA)

    - S2: numărul minim/maxim de componente tare conexe ale unui graf care permite sortare topologică (S2.3 2017CB)
- **Kosaraju**
    - S1: evidențiat componentele tare conexe (S1.2 2017CB)
- **Punți**
    - S3: găsirea punților pentru care se respectă o condiție (S3.6 2020)
    - S2: numărul minim/maxim de punți/puncte de articulație (S2.5 2017CA)
- **Componente biconexe**
    - S1: de menționat (S1.4 2020)
- **Drumuri minime**
    - S2: ce algoritm făcut la curs poate fi folosit pentru determinarea drumului de lungime maximă într-un graf orientat aciclic (S2.1 2017CA)
- **Dijkstra**
    - **S2: exemplu de ce nu merge decât pentru costuri pozitive (S2.4 2017CA)**
    - **S1: aplicat ++ evidențiat valoarea fiecărui nod din coadă și nodul extras (S1.3 2017CC)**
    - S2: modificarea pentru grafuri aciclice orientate (S2.1 restanță)
- **Bellman-Ford**
    - **S1: de calculat (S1.3 2019CB, S1.3 2017CA)**
- **Floyd-Warshall**
    - **S1: primii 2 pași + menționat matricile D(0), D(1), D(2) (S1.2 restanță)**
- **Kruskal**
    - **S1: de aplicat ++ menționat muchiile alese la fiecare pas (S1.4 2017CA, S1.3 2017CB)**
    - S2: demonstrat corectitudinea (S2.3 2017CC)
    - S2: asemănări, diferențe cu Prim (S2.2 restanță)
- **Prim**
    - **S1: găsirea unui arbore parțial de cost minim (S1.3 2020, S1.4 restanță)**
    - S2: semnificația vârfurilor d[v] + de ce muchiile alese sunt muchii sigure (S2.2 2019CB)
- **Flux maxim**
    - **S3: problema modificată cu A surse și B destinații (S3.3 2020)**
- **Edmonds-Karp**
    - **S1: de calculat (++ menționat drumurile de ameliorare și cantitatea maximă de flux ce poate fi transportată) (S1.4 2019CB, S1.5 2017CB, S1.5 2017CC, S1.3 restanță)**
    - S3: complexitate cu justificare (S2.4 2019CB)
- **A***
    - **S1: de aplicat (++ menționat cum se modifică OPEN și CLOSED la fiecare pas) cu o euristică dată (S1.5 2020, S1.5 2017CA, S1.4 2017CC, S1.5 restanță)**

    - S3: ce înseamnă admisibilitatea și dominanța + pe 8-puzzle 3 exemple de euristici nebanale dintre care una inadmisibilă + justificare de ce sunt așa (S2.5 2019CB)
    - S2: tot pe 8-puzzle dar numărul maxim de noduri care sunt scoase din CLOSED pentru a fi reintroduse în OPEN ++ principiul de la push-relabel ++ înălțimea maximă la care poate ajunge un nod în timpul rulării algoritmului (S2.4 2017CB)
    - S2: ce înseamnă admisibilitatea + ce se întâmplă dacă nu se respectă (S2.5 restanță)
- **Minimax, alfa-beta**
    - **S2: aplicat + evidențiat tăieturile alfa-beta + influențează ordinea de evaluare a nodurilor numărul de tăieturi dintr-un arbore de joc? (S2.2 2017CB)**
    - S2: aplicat + valoarea nodului A + evidențiat nodurile tăiate (S2.5 2017CC)
- Monte Carlo
    - S2: definirea complexității unui algoritm Monte Carlo (S2.2 2017CC)

Push-relabel

S2: numărul maxim de pompări saturate (S2.1 2017CC)